

Lesson 7 - Practical Application Guide

□ How to Run the Project

```
cd D:\learnlaravel2025\lessons\lesson-07\practice-app

# Setup database
copy .env.example .env
php artisan key:generate

# Run Migrations
php artisan migrate

# Run Seeders
php artisan db:seed

# Start server
php artisan serve
```

□ Implemented Relationships

1. One to One: User → Profile

Models:

- User.php: `hasOne(Profile::class)`
- Profile.php: `belongsTo(User::class)`

Usage:

```
$user = User::find(1);
echo $user->profile->bio;
echo $user->profile->website;
```

2. One to Many: User → Posts

Models:

- User.php: `hasMany(Post::class)`
- Post.php: `belongsTo(User::class)`

Usage:

```
$user = User::find(1);
foreach ($user->posts as $post) {
    echo $post->title;
}

$post = Post::find(1);
echo $post->user->name;
```

3. Many to Many: Posts ↔ Tags

Models:

- Post.php: `belongsToMany(Tag::class)`
- Tag.php: `belongsToMany(Post::class)`

Usage:

```
$post = Post::find(1);
$post->tags()->attach([1, 2, 3]);

foreach ($post->tags as $tag) {
    echo $tag->name;
}
```

4. Polymorphic: Comments on Posts/Videos

Models:

- Comment.php: `morphTo()`

- Post.php: `morphMany(Comment::class, 'commentable')`
- Video.php: `morphMany(Comment::class, 'commentable')`

Usage:

```
$post = Post::find(1);
$post->comments()->create(['content' => 'Great!']);

$comment = Comment::find(1);
$commentable = $comment->commentable;
```

Seeder Examples

```
// UserSeeder
User::factory()->create([
    'name' => 'John Doe',
    'email' => 'john@example.com',
]);

// ProfileSeeder
Profile::create([
    'user_id' => 1,
    'bio' => 'Laravel Developer',
    'website' => 'https://example.com',
]);

// PostSeeder
Post::create([
    'user_id' => 1,
    'title' => 'Learn Laravel',
    'content' => 'Post content...',
]);

// TagSeeder
$tags = ['Laravel', 'PHP', 'Web Development'];
foreach ($tags as $tag) {
    Tag::create(['name' => $tag, 'slug' => Str::slug($tag)]);
}

// Attach Tags to Posts
$post = Post::find(1);
```

```
$post->tags()->attach([1, 2, 3]);
```

▣ Usage Examples

Example 1: Eager Loading

```
// ▣ N+1 Problem
$post = Post::all();
foreach ($posts as $post) {
    echo $post->user->name; // Query per post
}

// ▣ Eager Loading
$post = Post::with('user')->get();
foreach ($posts as $post) {
    echo $post->user->name; // No additional queries
}

// Load multiple relationships
$post = Post::with(['user', 'comments', 'tags'])->get();

// Load nested
$post = Post::with('user.profile')->get();
$post = Post::with('comments.user')->get();
```

Example 2: withCount

```
// Count comments
$post = Post::withCount('comments')->get();
foreach ($posts as $post) {
    echo "{$post->title}: {$post->comments_count} comments";
}

// Count multiple relationships
$users = User::withCount(['posts', 'comments'])
    ->orderBy('posts_count', 'desc')
    ->get();
```

Example 3: whereHas

```
// Posts from active users
$post = Post::whereHas('user', function ($query) {
    $query->where('is_active', true);
})->get();

// Posts with approved comments
$post = Post::whereHas('comments', function ($query) {
    $query->where('approved', true);
})->get();

// Posts without comments
$post = Post::doesntHave('comments')->get();

// Posts with at least 5 comments
$post = Post::has('comments', '>=', 5)->get();
```

Example 4: Many to Many Operations

```
$post = Post::find(1);

// Attach tags
$post->tags()->attach(1);
$post->tags()->attach([1, 2, 3]);
$post->tags()->attach(1, ['order' => 1]); // With additional data

// Detach tags
$post->tags()->detach(1);
$post->tags()->detach([1, 2]);
$post->tags()->detach(); // Remove all

// Sync (replace)
$post->tags()->sync([1, 2, 3]);

// Sync without detaching
$post->tags()->syncWithoutDetaching([4, 5]);

// Toggle
$post->tags()->toggle([1, 2]);
```

```
// Check
if ($post->tags->contains(1)) {
    echo "Has tag";
}
```

Example 5: Polymorphic Comments

```
// Comment on post
$post = Post::find(1);
$comment = $post->comments()->create([
    'content' => 'Great post!',
]);

// Comment on video
$video = Video::find(1);
$comment = $video->comments()->create([
    'content' => 'Nice video!',
]);

// Get item from comment
$comment = Comment::find(1);
$commentable = $comment->commentable;

if ($commentable instanceof Post) {
    echo "Comment on post: " . $commentable->title;
} elseif ($commentable instanceof Video) {
    echo "Comment on video: " . $commentable->title;
}
```

▣ Testing with Tinker

```
php artisan tinker
```

```
// Test One to One
$user = User::find(1);
$user->profile;
$user->profile->bio;
```

```
// Test One to Many
$user->posts;
$user->posts()->count();

// Test Many to Many
$post = Post::find(1);
$post->tags;
$post->tags()->attach(1);
$post->tags()->sync([1, 2, 3]);

// Test Eager Loading
Post::with('user')->get();
Post::with(['user', 'comments', 'tags'])->get();

// Test withCount
Post::withCount('comments')->get();
User::withCount(['posts', 'comments'])->get();

// Test whereHas
Post::whereHas('tags', function ($query) {
    $query->where('name', 'Laravel');
})->get();

// Test Polymorphic
$post = Post::find(1);
$post->comments()->create(['content' => 'Test']);
$comment = Comment::find(1);
$comment->commentable;
```

❑ Important Tips

❑ Best Practices

1. Always use Eager Loading to avoid N+1 problem
2. Use withCount for statistics instead of count()
3. Use whereHas to query relationships
4. Specify required columns: `with('user:id,name')`
5. Use Soft Deletes with `onDelete('cascade')`

⚠ Common Mistakes

1. Forgetting Eager Loading (N+1 problem)
 2. Using `get()` twice
 3. Forgetting to define `belongsTo` on the other side
 4. Not adding Foreign Keys in Migrations
 5. Forgetting `unique()` in Pivot Tables
-

📄 Useful Commands

```
# Create Models with Migrations
php artisan make:model Post -m
php artisan make:model Profile -m

# Create Seeder
php artisan make:seeder PostSeeder

# Run Migrations
php artisan migrate
php artisan migrate:fresh --seed

# Tinker
php artisan tinker
```

📄 Next Step

After completing this lesson, you're ready for:

Lesson 8: Validation and Form Requests

- Basic Validation
 - Form Request Classes
 - Custom Validation Rules
-

Happy Learning! ☺